To begin with, you can omit the use of macro *gen_context()*. We will cover macros in a later part of this series. After that, we will revisit the *gen_context()* macro.

For our current purpose, let's create the file, *test.fc*, containing the following line:

```
/var/www/html/index.html --     system_u:object_r:lfy_t:s0
```

...and recompile our policy module. Do remember to increase the version number in the type enforcement file, *test.te*.

Let us update our test module:

```
[root@vbg-work ~]# semodule -u /home/vbg/test-selinux/test.pp
```

To see the effect of our new *test.fc* file, let's check the current context of the file */var/www/html/index.html* and then restore it to its default context:

```
[root@vbg-work ~]# ls -lZ /var/www/html/index.html
-rw-r--r--. root root system_u:object_r:httpd_sys_content_ts0 /var/www/
html/index.html

[root@vbg-work ~]# restorecon -v /var/www/html/index.html
restorecon reset /var/www/html/index.html context system_u:object_r:
httpd_sys_content_ts0->system_u:object_r:lfy_t:s0

[root@vbg-work ~]# ls -lZ /var/www/html/index.html
-rw-r--r--. root root system_u:object_r:lfy_t:s0      /var/www/html/index.html

[root@vbg-work ~]#
```

Now we can see that the default context of the file has been set to *lfy_t*. What context does a newly created file get? Let us test this by creating a new file */var/www/html/test.html* by using the *touch* command:

```
[root@vbg-work html]# touch /var/www/html/test.html
[root@vbg-work html]# ls -lZ /var/www/html/test.html
-rw-r--r--. root root unconfined_u:object_r:httpd_sys_content_ts0 /var/
www/html/test.html
[root@vbg-work html]#
```

This shows that the default security context of the created file takes the type *httpd_sys_content_t*. Why is it so?

Let us create a new folder under */var/www/html/*:

```
[root@vbg-work html]# mkdir /var/www/html/test
[root@vbg-work html]# ls -lZ /var/www/html
-rw-r--r--. root root system_u:object_r:lfy_t:s0      index.html
drwxr-xr-x. root root unconfined_u:object_r:httpd_sys_content_ts0 test
-rw-r--r--. root root unconfined_u:object_r:httpd_sys_content_ts0 test.html
[root@vbg-work html]#
```

The newly created folder also gets a security context with the type *httpd_sys_content_t*.

Now let us set the type of this new folder to *lfy_t* and then create new files under it:

```
[root@vbg-work html]# chcon -t lfy_t /var/www/html/test
[root@vbg-work html]# touch /var/www/html/test/test1.html
[root@vbg-work html]# ls -lZ /var/www/html/test/
-rw-r--r--. root root unconfined_u:object_r:lfy_t:s0   test1.html
[root@vbg-work html]#
```

Surprise! The new files are of type *lfy_t*. This tells us about the default behaviour when applying security contexts to newly created files. As observed above, newly created files take the security context of the parent folder. Is it really so? Let us test this with a few more examples.

Security context of the root folder is *root_t*:

```
[root@vbg-work html]# ls -lZd /
dr-xr-xr-x. root root system_u:object_r:root_t:s0      /
```

Create a new file under the root folder and check its security context. Going by the above logic, the type of a newly created file (for example, *test.txt*) should be *root_t*:

```
[root@vbg-work html]# touch /test.txt
[root@vbg-work html]# ls -lZ /test.txt
-rw-r--r--. root root unconfined_u:object_r:etc_runtime_t:s0 /test.txt
[root@vbg-work html]#
```

What happened? The file *test.txt* under folder / (type = *root_t*) was supposed to bear a security context with type *root_t*. Instead, its type is *etc_runtime_t*.

Why is the type of the file not *root_t*. Why and how did the type of this file change? This is achieved through Type Transition. When a new object is created, we can create a rule in our policy module that can change the type of the newly-created object. If no rule is present, the newly-created object inherits the type of its parents.

A new object is created by a process (a *subject*, in SELinux terminology). In the above example, when we created a new file */text.txt*, we did so by using the *touch* command. This was executed on the terminal windows (bash process). Let us check the type of the bash process:

```
[root@vbg-work html]# ps axZ | grep bash
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 1896 pts/0 Ss+  0:00
bash
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 2152 pts/1 Ss   0:00 bash
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 2193 pts/2 Ss   0:00 bash
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 2216 pts/3 Ss   0:00 bash
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 2241 pts/3 S    0:00 bash
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 2372 pts/4 Ss   0:00 bash
```

It does not matter which one of the above processes we used while creating the file, because all the processes have the type *unconfined_t*. Our policy could have had a rule which said: